Technical Architecture

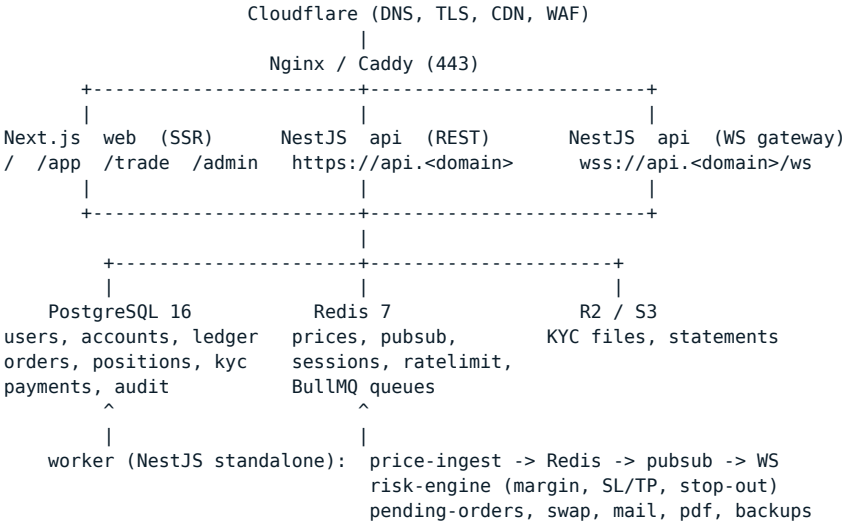
Spova-style CFD platform | marketing + client portal + trading terminal + admin | one API, one database

This document is the complete build spec. If something is not in here, it is not in scope for 5-16 Oct 2026. Each section is a checklist: routes, endpoints, events, tables, flows, jobs, infra, env, and acceptance per milestone.

#	Section	What it locks down
01	System overview	Components, boundaries, data flow, what is on our books
02	Monorepo layout	Folders, packages, shared types, where code lives
03	Frontend route map	Every page in / , /app , /trade , /admin with API calls
04	REST API	Every endpoint, method, auth, payload summary
05	WebSocket events	Rooms, client->server, server->client, payloads
06	Database schema	Every table, column, type, enum, index
07	Core flows	Auth, KYC, deposit, withdraw, credit/debit, open/close, SL/TP, stop-out, freeze
08	Risk + pricing engine	Formulas, tick loop, candles, Redis keys, pending orders, swap
09	Security	Cookies, RBAC, rate limits, uploads, audit, secrets
10	Jobs + queues	BullMQ queues, schedules, retries
11	Infra + deploy	Docker Compose, Nginx, domains, backups, monitoring, CI
12	Environment variables	Full .env list for api / web / worker
13	Milestone acceptance	What must exist on 9 / 13 / 16 Oct
14	Do-not-miss list	Edge cases that break broker software

01 SYSTEM OVERVIEW

Four user-facing surfaces, one NestJS API, one PostgreSQL. Prices come from an external feed worker into Redis, are fanned out over WebSocket, and drive a server-side risk engine. We are the counterparty (CFD on our books). Money never moves except through LEDGER_ENTRY inside a DB transaction.



Boundaries

Component	Owns	Does NOT own
web (Next.js)	Rendering, forms, charts, client WS subscriptions, cookie forwarding	Business rules, balance math, price source of truth
api (NestJS)	Auth, RBAC, validation, ledger, orders, admin ops, WS fanout	Long loops (those live in worker)
worker	Price ingest, candles, risk engine, pending orders, swap, mail, PDF	HTTP routes
PostgreSQL	All durable state, money, audit	Caching, live prices
Redis	Last price per symbol, pubsub, BullMQ, rate limit, refresh-token denylist	Durable state
R2	KYC documents, statement PDFs (private bucket, signed URLs)	Anything public

Trading model v1

- Instant execution at current bid/ask from Redis. No order book, no external LP. Spread = feed spread + optional markup per symbol.
- Position P/L recalculated server-side on each tick for accounts with open positions; pushed to the owner over WS.
- Margin call at 100% (notice), stop-out at 20% (force-close worst position first, repeat until level > stop-out).
- Demo and live accounts share every table and screen. Difference is type=demo|live, withdrawal allowed only on live.
- Deposits v1 = request + admin approve (ledger credit). Gateway adapter interface exists; no PSP connected yet.

02 MONOREPO LAYOUT

```
spova/
  apps/
    web/                    Next.js 15 (App Router) - all four surfaces
      app/(marketing)/      /  /forex /commodities /indices /shares /crypto /metals
      app/(auth)/           /login /register /forgot-password /reset-password
      app/(client)/app/     client portal (rail layout)
      app/(trade)/trade/    terminal (full-bleed layout)
      app/(admin)/admin/    back office (sidebar layout, role guard)
      components/           ui (shadcn), charts, forms, tables
      lib/api.ts            typed fetch wrapper (credentials: include)
      lib/ws.ts             socket.io client, reconnect, room subscribe
      middleware.ts         cookie presence + role redirect
    api/                    NestJS
      src/modules/          auth users kyc accounts ledger payments market trading risk admin support statements realtime
      src/common/           guards, decorators, filters, pipes, dto base, money utils
      prisma/schema.prisma  single schema
      prisma/seed.ts        admin user, symbols, risk config, demo account
```

```
  worker/                  NestJS standalone (no HTTP)
    src/price-ingest/      binance ws, (later) twelvedata ws -> Redis + pubsub
    src/candles/           tick -> 1m/5m/15m/1h/4h/1d aggregation -> candles table
    src/risk-engine/       margin loop, SL/TP, stop-out, margin-call notices
    src/pending-orders/    limit/stop matcher
    src/jobs/              swap, mail, statement-pdf, backup
packages/
  shared/                  zod schemas + TS types shared web<->api (DTOs, enums, WS payloads)
  config/                  eslint, tsconfig, tailwind preset
infra/
  docker-compose.yml       postgres redis api worker web caddy
  Caddyfile                TLS + reverse proxy
  backup.sh                pg_dump -> R2 nightly
docs/                      PLATFORM_PLAN.md/.pdf  ARCHITECTURE.pdf  quotations
.github/workflows/ci.yml   lint, typecheck, prisma validate, build
```

Rule: DTOs and enums live in packages/shared and are imported by both web and api. No duplicated types.

03 FRONTEND ROUTE MAP

Marketing (public, SSR, SEO)

Route	Page content	Data
/	Hero, instrument tabs (FX/Indices/Stocks/Commodities/Crypto/Metals), advantages, education, CTA, footer with risk warning	GET /market/symbols (featured) | static copy
/forex /commodities /indices /shares /cryptocurrencies /metals	Market intro + instrument cards (symbol, name, leverage up to, spread from)	GET /market/symbols?class=
/trading-account	Tier comparison table Classic/Silver/Gold/Platinum/VIP	GET /market/risk-config (public subset)
/classic-account ... /vip-account	Single tier detail + compare table	same
/platform	WebTrader marketing	static
/education /faq /about-us /contact-us /legal /complaints	Content pages (MDX), contact form -> ticket	POST /support/contact
/cfds-list /swap-fees /market-holidays /cfd-expiries	Tables from symbols + sessions	GET /market/symbols, /market/sessions

Auth

Route	Page	API
/login	Email + password, show/hide, Open account, Forgot password, risk warning	POST /auth/login
/register	Step 1 email/password/phone/country, step 2 personal (name, dob, address), T&C; + risk accept	POST /auth/register
/forgot-password	Email -> link	POST /auth/forgot-password
/reset-password?token=	New password + confirm, policy hint	POST /auth/reset-password
/verify-email?token=	Optional in v1	POST /auth/verify-email

Client portal /app (left rail like Spova, content as modal/panels)

Route	Page	API
/app	Dashboard: account switcher (demo/live chip), balance/equity/free margin cards, KYC status banner, quick Deposit, open positions summary, recent payments	GET /me, /me/accounts, /me/kyc/status, /me/payments?limit=5 | WS account
/app/accounts	Live accounts (Open live account CTA), Demo accounts cards: login code, equity, leverage, balance, free margin; 'Trade' button -> /trade?acc=	GET /me/accounts | POST /me/accounts
/app/payments/deposit	Tabs Deposit/Withdrawal/History. Deposit: amount presets, method (Admin transfer / Crypto USDT placeholder / Card coming soon), submit request	GET /me/payments/methods | POST /me/payments/deposit
/app/payments/withdraw	Amount, available, reason, fee note; blocked on demo; requires KYC approved on live	POST /me/payments/withdraw
/app/payments/history	List: kind, id, date, status chip, amount; filter	GET /me/payments
/app/documents	Groups POI / POR / Payment / Other, each: type select, upload, cards with status Pending/Approved/Rejected + reason	GET /me/kyc/*, POST /me/kyc/upload-url, POST /me/kyc/documents
/app/statement	Account + date range -> Export PDF	GET /me/accounts/:id/statement.pdf
/app/help	Ticket form (subject, message, cc me) + my tickets list	GET/POST /me/tickets
/app/profile	Tabs Personal / Settings / Password	GET/PATCH /me/profile, /me/notification-prefs, PATCH /me/password

Trading terminal /trade

Area	Component	Data
Top bar	Account selector, balance | equity | margin | free margin | margin level (color), server time, user menu	WS account
Left	Watchlist: search, groups by class, bid/ask, change %, click -> chart	GET /market/symbols | WS tick

Area	Component	Data
Center	Lightweight Charts candlestick, timeframe 1m 5m 15m 1h 4h 1d, crosshair, position lines (entry/SL/TP), last price line	GET /market/candles | WS candle + tick
Right	Order ticket: Buy/Sell, lots (+/- step min_lot), SL/TP (price or pips), required margin preview, est. spread; Market | Limit | Stop	POST /me/accounts/:id/orders
Bottom	Tabs Open positions / Pending orders / History. Columns: id, symbol, side, lots, open, current, SL, TP, swap, P/L, close X, edit	GET positions/orders | WS position:*
Modals	Edit SL/TP, partial close, confirm market order, margin call toast, stop-out alert, trading blocked banner	PATCH / POST close

Admin /admin (sidebar, role=admin|superadmin)

Route	Page	API
/admin	KPIs: clients, pending KYC, pending payments, open positions, total exposure per symbol, equity at risk	GET /admin/dashboard
/admin/clients	Table search/filter (status, kyc, country), row -> detail	GET /admin/users
/admin/clients/:id	Tabs: Profile | KYC docs (approve/reject) | Accounts (leverage, tier, status, trading block, credit/debit) | Ledger | Payments | Positions (force close) | Tickets | Audit	GET /admin/users/:id + sub-resources
/admin/kyc	Queue by status; preview file via signed URL; approve / reject with reason	GET /admin/kyc, PATCH /admin/kyc/:id
/admin/payments	Deposit/withdraw requests; approve (ledger) / decline (note)	GET/PATCH /admin/payments
/admin/wallet	Pick account -> credit / debit / bonus credit with reason; shows balance after	POST /admin/accounts/:id/credit|debit
/admin/trades	All open positions live, filter symbol/account; force close; pending orders	GET /admin/positions | WS admin
/admin/symbols	Enable/disable, spread markup, leverage_max, lot limits, sessions	GET/PATCH /admin/symbols
/admin/risk	Per-tier margin call %, stop-out %, default leverage, max leverage	GET/PATCH /admin/risk-config
/admin/tickets	Inbox, reply, close	GET/POST /admin/tickets
/admin/audit	Who did what to whom, when, before/after JSON	GET /admin/audit
/admin/staff	superadmin only: create admin, reset, disable	POST /admin/staff

04 REST API (prefix /v1, JSON, cookie auth)

Conventions: 2xx JSON, errors {code, message, details}. Money as string decimals. IDs uuid. Pagination ?page&limit; -> {items,total}. All write endpoints accept Idempotency-Key header (stored 24h in Redis).

Method + path	Auth	Purpose / payload
POST /auth/register	public	{email,password,phone,country,firstName,lastName,dob,tcAccepted,riskAccepted} -> sets cookies, creates demo account
POST /auth/login	public	{email,password} -> access (15m) + refresh (7d) httpOnly cookies; rate limited 5/min/IP
POST /auth/refresh	refresh cookie	rotate refresh, new access; old refresh denylisted
POST /auth/logout	user	clear cookies, denylist refresh
POST /auth/forgot-password	public	{email} always 200; emails signed token (1h)
POST /auth/reset-password	public	{token,password}
GET /auth/me	user	{id,email,role,status,kyc:{poi,por},flags}
GET|PATCH /me/profile	user	name, phone, dob, country, city, address, zip (email not editable)
PATCH /me/password	user	{current,next}
GET|PATCH /me/notification-prefs	user	{email,sms,push,calls}
GET /me/kyc/status	user	{poi,por,payment:'incomplete|pending|approved|rejected'}
GET /me/kyc/documents	user	list with status + rejectReason
POST /me/kyc/upload-url	user	{group,type,mime,size} -> {uploadUrl,fileKey} (R2 presigned PUT, max 10MB, jpg/png/pdf)
POST /me/kyc/documents	user	{group,type,fileKey,originalName} -> pending; notifies admin room
GET /me/accounts	user	demo[] live[] with balance, credit, equity, usedMargin, freeMargin, leverage, status
POST /me/accounts	user	{type:'live'|'demo', currency} ; live requires KYC approved (configurable)
GET /me/accounts/:id/ledger	owner	paginated ledger entries
GET /me/accounts/:id/statement.pdf?from&to;	owner	generates (queued) and streams PDF
GET /me/payments	user	requests + approved ledger items, filter kind/status
GET /me/payments/methods	user	enabled methods + min/max from config
POST /me/payments/deposit	user	{accountId,amount,method,reference?} -> PAYMENT_REQUEST pending
POST /me/payments/withdraw	user	{accountId,amount,reason?,destination} ; live only, KYC approved, amount <= free margin - open margin
GET /market/symbols?class=	public	symbol specs + last quote
GET /market/symbols/:id	public	spec incl digits, contractSize, minLot, maxLot, leverageMax, sessions
GET /market/candles/:symbol?tf=1m&from;&to;&limit;	public	ohlc array (max 1500)
GET /market/quotes?symbols=	public	snapshot bid/ask/ts
GET /market/risk-config	public	tier table (marginCall, stopOut, leverage)
GET /me/accounts/:id/positions?status=open|closed	owner	positions with live profit (open) from Redis price
GET /me/accounts/:id/orders?status=	owner	pending/filled/cancelled/rejected
POST /me/accounts/:id/orders	owner	{symbol,side,type:'market'|'limit'|'stop',lots,price?,sl?,tp?} -> market fills now, else pending
DELETE /me/accounts/:id/orders/:oid	owner	cancel pending
PATCH /me/accounts/:id/positions/:pid	owner	{sl?,tp?} validated vs side and current price
POST /me/accounts/:id/positions/:pid/close	owner	{lots?} partial or full at current price -> ledger pnl
GET|POST /me/tickets, GET /me/tickets/:id, POST /me/tickets/:id/messages	user	support
GET /admin/dashboard	admin	counts + exposure

Method + path	Auth	Purpose / payload
`GET /admin/users?q&status;&kyc;&page;`	admin	list
`GET /admin/users/:id`	admin	full profile + accounts + kyc + flags
`PATCH /admin/users/:id`	admin	{status:'active'|'frozen'|'closed', notes}
`POST /admin/users/:id/reset-password-link`	admin	send reset mail
`GET /admin/kyc?status`	admin	queue
`GET /admin/kyc/:docId/file`	admin	signed GET URL (5 min)
`PATCH /admin/kyc/:docId`	admin	{status:'approved'|'rejected',reason?} ; updates user.kyc_poi/por when group complete
`GET /admin/accounts?userId&type;`	admin	list
`PATCH /admin/accounts/:id`	admin	{leverage?,tier?,status?,tradingBlocked?} ; leverage change recalcs margin
`POST /admin/accounts/:id/credit`	admin	{amount,reason,type:'deposit'|'credit'|'bonus'} -> ledger + balance in one tx
`POST /admin/accounts/:id/debit`	admin	{amount,reason,type:'withdraw'|'debit'|'fee'} ; cannot push free margin < 0 unless force
`GET /admin/accounts/:id/ledger`	admin	ledger
`GET /admin/payments?status&kind;`	admin	queue
`PATCH /admin/payments/:id`	admin	{status:'approved'|'declined',note?} ; approve = ledger entry, ref_id = payment id
`GET /admin/positions?open=1&symbol;&accountId;`	admin	live list
`POST /admin/positions/:id/close`	admin	force close, reason, audit
`GET /admin/orders?status`	admin	pending orders
`GET|PATCH /admin/symbols/:id`	admin	enable, spreadMarkup, leverageMax, minLot, maxLot, sessions
`GET|PATCH /admin/risk-config`	admin	per tier
`GET /admin/tickets, POST /admin/tickets/:id/messages, PATCH /admin/tickets/:id`	admin	support
`GET /admin/audit?actorId&targetId;&action;`	admin	audit log
`POST /admin/staff, PATCH /admin/staff/:id`	superadmin	manage admins
`GET /health, GET /health/ready`	public	liveness (db, redis)

05 WEBSOCKET EVENTS (socket.io namespace /ws, cookie auth on handshake)

Rooms

Room	Who joins	Purpose
`symbol:{SYMBOL}`	any authenticated client that subscribes	ticks + candle updates
`user:{userId}`	auto on connect	account snapshots, position changes, kyc/payment status
`account:{accountId}`	owner (selected account)	fine-grained account/position stream
`admin`	role admin|superadmin	new KYC, new payment request, position opened/closed, stop-outs

Client -> Server

Event	Payload	Notes
`subscribe`	`{symbols:string[]}`	max 50 symbols; joins symbol rooms
`unsubscribe`	`{symbols:string[]}`	
`select_account`	`{accountId}`	joins account room, leaves previous
`ping`	`{}`	heartbeat; server replies pong with server ts

Server -> Client

Event	Payload	Frequency
`tick`	`{s,bid,ask,ts}`	throttled to <= 4/s per symbol
`candle`	`{s,tf,t,o,h,l,c,v?}`	on each tick for live bar (1m) + on bar close for all tf
`account`	`{accountId,balance,credit,equity,usedMargin,freeMargin,marginLevel,ts}`	on change, max 2/s
`position:opened | position:updated | position:closed`	`full position row (+closeReason: user|sl|tp|stopout|admin)`	event
`order:filled | order:cancelled | order:rejected`	`order row (+reason)`	event
`notice`	`{type:'margin_call'|'stop_out'|'trading_blocked'|'kyc'|'payment',title,body,level}`	event
`kyc:status`	`{group,status,reason?}`	event
`payment:status`	`{paymentId,status,note?}`	event
`admin:kyc:new | admin:payment:new | admin:position:event | admin:stopout`	`minimal row`	admin room only

Pricing fanout: worker publishes ticks to Redis channel ticks; api WS gateway subscribes once and emits to symbol rooms. Web never talks to the feed directly.

06 DATABASE SCHEMA (PostgreSQL 16, Prisma)

Conventions: id uuid default gen_random_uuid(); created_at/updated_at timestamptz; money NUMERIC(18,2); prices NUMERIC(18,6); lots NUMERIC(10,2). Soft delete only on users (status=closed). All enums are Postgres enums.

Enums

```
role            client | admin | superadmin
user_status     active | frozen | closed
account_type    demo | live
account_status  enabled | disabled | readonly
tier            classic | silver | gold | platinum | vip
ledger_type     deposit | withdraw | credit | debit | bonus | pnl | swap | fee | commission | adjustment
payment_kind    deposit | withdraw
payment_status  pending | approved | declined | cancelled
kyc_group       poi | por | payment | other
kyc_status      pending | approved | rejected
asset_class     forex | index | commodity | stock | crypto | metal
order_side      buy | sell
order_type      market | limit | stop
order_status    pending | filled | cancelled | rejected | expired
position_status open | closed
close_reason    user | sl | tp | stopout | admin | expiry
ticket_status   open | answered | closed
timeframe       m1 | m5 | m15 | h1 | h4 | d1
```

users

column	type	notes
id	uuid PK	
email	citext UNIQUE	login
password_hash	text	argon2id
role	role	default client
status	user_status	frozen blocks login + trading
email_verified_at	timestamptz?	
kyc_poi / kyc_por / kyc_payment	bool x3	derived when group approved
tc_accepted_at / risk_accepted_at	timestamptz	register
last_login_at / last_login_ip		
referral_code / referred_by	text?	partner later
created_at / updated_at		

user_profiles

column	type	notes
user_id	uuid PK FK	
first_name / last_name	text	
phone	text	E.164
dob	date	18+ validation
country_code	char(2)	ISO
city / address / zip	text	
language	text	default en
notif_email / notif_sms / notif_push / notif_calls	bool	prefs

sessions

column	type	notes
id	uuid PK	refresh token id (jti)
user_id	uuid FK idx	
token_hash	text	sha256 of refresh
user_agent / ip	text	
expires_at	timestamptz	
revoked_at	timestamptz?	rotation / logout

password_resets

column	type	notes
id	uuid	
user_id	uuid FK	
token_hash	text	
expires_at / used_at	timestamptz	

trading_accounts

column	type	notes
id	uuid PK	
user_id	uuid FK idx	
type	account_type	
login_code	int UNIQUE	6-digit shown to user
tier	tier	default classic
currency	char(3)	USD v1
leverage	int	<= risk_config.max
balance	numeric(18,2)	realized cash
credit	numeric(18,2)	bonus/credit, not withdrawable
status	account_status	
trading_blocked	bool	admin stop trade
margin_call_at	timestamptz?	last notice
created_at / updated_at		

equity/used_margin/free_margin are computed, never stored as truth. Row lock (SELECT ... FOR UPDATE) on every money change.

ledger_entries

column	type	notes
id	uuid PK	
account_id	uuid FK idx	
type	ledger_type	
amount	numeric(18,2)	signed
balance_after	numeric(18,2)	snapshot
credit_after	numeric(18,2)	snapshot
reason	text	human text
ref_type / ref_id	text / uuid	payment_request | position | admin
actor_id	uuid?	admin or null=system
created_at	timestamptz idx	append-only; no UPDATE/DELETE grants

payment_requests

column	type	notes
id	uuid PK	
account_id / user_id	uuid FK	
kind	payment_kind	
amount	numeric(18,2)	
currency	char(3)	
method	text	admin_transfer | crypto_usdt | card | bank
destination	jsonb?	wallet / bank for withdraw
reference	text?	user txid / note
status	payment_status	
admin_note	text?	
decided_by / decided_at	uuid? / timestamptz?	
ledger_entry_id	uuid?	set on approve
created_at		idx status

kyc_documents

column	type	notes
id	uuid PK	
user_id	uuid FK idx	
group	kyc_group	
doc_type	text	poi_passport, poi_id_front, poi_id_back, poi_dl_front, poi_dl_back, por_utility, por_bank, por_residence, pay_card_front, pay_bank, other
file_key	text	R2 key, private
original_name / mime / size		validated
status	kyc_status	
reject_reason	text?	
reviewed_by / reviewed_at		
created_at		

symbols

column	type	notes
id	text PK	EURUSD, XAUUSD, BTCUSD, US500, AAPL
name	text	
asset_class	asset_class	
digits	int	price decimals
contract_size	numeric	100000 fx, 1 crypto, 100 oz gold...
min_lot / max_lot / lot_step	numeric(10,2)	0.01 / 50 / 0.01
leverage_max	int	400 fx, 200 idx, 5 stock/crypto
spread_markup	numeric(18,6)	added to feed spread
swap_long / swap_short	numeric(18,6)	per lot per night
feed_source / feed_symbol	text	binance:btcusdt, twelvedata:EUR/USD
enabled / is_open	bool	admin + session calc
sort_order / featured		marketing

symbol_sessions

column	type	notes
symbol_id	text FK	
weekday	int 0-6	
open_time / close_time	time	UTC
holiday dates	symbol_holidays(symbol_id, date)	separate table

candles

column	type	notes
symbol_id	text	PK(symbol_id, tf, t)
tf	timeframe	
t	timestamptz	bar open
o/h/l/c	numeric(18,6)	
v	numeric?	ticks count

Retention: m1 30 days, m5 90 days, others 3 years. Partition by month if volume grows.

orders

column	type	notes
id	uuid PK	
account_id	uuid FK idx	
symbol_id	text FK	
side / type	enums	
lots	numeric(10,2)	
price	numeric(18,6)?	limit/stop trigger
sl / tp	numeric(18,6)?	
status	order_status	
filled_price / filled_at		
position_id	uuid?	
reject_reason	text?	
expires_at	timestamptz?	GTC default
client_ref	text?	idempotency
created_at		

positions

column	type	notes
id	uuid PK	
account_id	uuid FK idx(status)	
symbol_id	text FK	
side	order_side	
lots	numeric(10,2)	remaining after partial
open_price	numeric(18,6)	
open_at	timestamptz	
sl / tp	numeric(18,6)?	
margin_used	numeric(18,2)	at open; recalculated on leverage change
swap_accrued	numeric(18,2)	
commission	numeric(18,2)	
status	position_status	
close_price / close_at		
realized_pnl	numeric(18,2)?	
close_reason	close_reason?	
closed_by	uuid?	admin id if forced

position_closes

column	type	notes
id	uuid	
position_id	uuid FK	
lots	numeric	partial amount
price	numeric	
pnl	numeric(18,2)	
reason	close_reason	
ledger_entry_id	uuid	
created_at		

risk_configs

column	type	notes
tier	tier PK	
margin_call_pct	numeric(5,2)	100
stop_out_pct	numeric(5,2)	20
default_leverage / max_leverage	int	100 / 400
min_deposit	numeric(18,2)	150
spread_discount_pct	numeric	tier perk

tickets + ticket_messages

column	type	notes
tickets	id, user_id, subject, status, assigned_to, created_at, updated_at	
ticket_messages	id, ticket_id, author_id, body, is_staff, created_at	

admin_audit

column	type	notes
id	uuid	
actor_id	uuid FK	admin
action	text	user.freeze, account.credit, kyc.approve, position.force_close, symbol.update ...
target_type / target_id	text / uuid	
before / after	jsonb	diff
ip	inet	
created_at	timestamptz idx	append-only

notifications + statement_exports + idempotency_keys

column	type	notes
notifications	id, user_id, type, title, body, read_at, created_at	in-app bell
statement_exports	id, account_id, from, to, file_key, status, created_at	PDF jobs
idempotency_keys	key PK, user_id, response jsonb, expires_at	or Redis only

07 CORE FLOWS

Register -> first login

- POST /auth/register validates (zod), checks country not blocked, hashes argon2id, creates users + user_profiles + demo trading_account (balance from config, e.g. 10,000) + ledger 'credit' entry.
- Sets access + refresh cookies (httpOnly, Secure, SameSite=Lax, Domain=.). Returns /auth/me shape.
- Web middleware: no access cookie -> redirect /login; role=admin visiting /app allowed; client visiting /admin -> 403 page.

Token refresh

- Access 15 min. Web api wrapper on 401 calls /auth/refresh once then retries. Refresh rotates: old jti revoked in sessions, denylisted in Redis until expiry. Reuse of revoked refresh = revoke all sessions for user (theft signal).

KYC upload -> approve

- Client requests presigned PUT (mime/size checked), uploads to R2 directly, then POST /me/kyc/documents with fileKey. Status pending. WS admin:kyc:new.
- Admin previews via 5-min signed GET. PATCH approve/reject (+reason). On approve, if group has required types satisfied, set users.kyc_poi/por. WS kyc:status to user. Audit row.
- Withdrawals and live account creation check users.kyc_poi AND kyc_por (toggle in config).

Deposit request -> approve (ledger credit)

- POST /me/payments/deposit -> payment_requests pending (min/max per method). WS admin:payment:new, email to admin.
- PATCH /admin/payments/:id approved -> TRANSACTION: SELECT account FOR UPDATE; INSERT ledger_entries(type=deposit, amount, balance_after=balance+amount, ref=payment); UPDATE account.balance; UPDATE payment status + ledger_entry_id; INSERT admin_audit. COMMIT. WS account + payment:status.
- Declined -> status + note, no ledger. Cancelled by user only while pending.

Withdraw request -> approve (ledger debit)

- Preconditions: account.type=live, user KYC approved, status active, amount <= free_margin and <= balance (credit not withdrawable). Lock amount? v1: no hold; re-check at approval.
- Approve: same transaction pattern with type=withdraw negative amount; reject if re-check fails (free margin changed).

Admin credit / debit

- POST /admin/accounts/:id/credit {amount,reason,type}: type deposit|credit|bonus. bonus -> account.credit; others -> balance. Same locked transaction + ledger + audit.
- Debit cannot make equity negative unless force=true (superadmin). Always writes reason.

Open market position

- Validate: account enabled, not trading_blocked, user active, symbol enabled and session open, lots within min/max/step.
- Price: Redis price:{symbol}; buy at ask, sell at bid; apply spread_markup. Reject if quote older than 10s.
- required_margin = lots * contract_size * price / leverage (converted to account currency). Reject if free_margin < required_margin.
- TRANSACTION: lock account; INSERT orders(filled) + positions(open, margin_used); commission ledger if any. WS order:filled, position:opened, account.

Pending limit/stop order

- Stored pending with margin pre-check only (not reserved). Worker pending-orders checks on each tick of that symbol: limit buy fills when ask <= price; limit sell when bid >= price; stop buy when ask >= price; stop sell when bid <= price. Fill path = same as market open with fill price = trigger-side quote. Re-check margin at fill; reject if insufficient.

SL / TP and close

- PATCH sl/tp: validate side-correct (buy: sl < price < tp). Worker risk-engine checks on each tick: buy closes at bid; sell at ask.
- Close (user/sl/tp/admin): pnl = (close - open) * lots * contract_size * (buy ? +1 : -1) [+ swap_accrued - commission]. TRANSACTION: lock account; UPDATE position closed; INSERT position_closes; INSERT ledger(type=pnl); UPDATE balance. WS position:closed + account.
- Partial close: reduce lots and margin_used proportionally, ledger pnl for the closed part.

Margin engine tick

- On each tick for symbol S: for every open position on S recompute floating pnl (cached in Redis pos:{id}). For each affected account compute equity, used_margin, free_margin, margin_level = equity/used_margin*100 (used_margin>0).
- level <= margin_call_pct and not notified in last 1h -> notice margin_call, set margin_call_at. level <= stop_out_pct -> close positions by largest loss first until level > stop_out_pct or no positions. close_reason=stopout. WS notice stop_out + admin:stopout.

Freeze / block

- PATCH /admin/users/:id status=frozen: revoke sessions, block login. PATCH account tradingBlocked=true: open/close by user rejected (admin can still force close), deposits still allowed, withdrawals blocked. Both audit + WS notice.

Leverage change by admin

- Recompute margin_used for open positions with new leverage; if resulting margin_level < stop_out, reject the change (or force with warning). Audit before/after.

08 RISK + PRICING ENGINE

Formulas (account currency USD; quote currency conversion via USD pairs when needed)

```
notional          = lots * contract_size * price
required_margin   = notional / leverage                        (cap leverage at symbol.leverage_max)
floating_pnl(buy) = (bid - open_price) * lots * contract_size
floating_pnl(sell)= (open_price - ask) * lots * contract_size
equity            = balance + credit + sum(floating_pnl) + sum(swap_accrued) - sum(commission_open)
used_margin       = sum(position.margin_used)
free_margin       = equity - used_margin
margin_level_pct  = used_margin > 0 ? equity / used_margin * 100 : null
margin_call       : margin_level_pct <= risk_config[tier].margin_call_pct   (default 100)
stop_out          : margin_level_pct <= risk_config[tier].stop_out_pct      (default 20)
pip (fx 5-digit)  = 0.0001 ; pip value per lot = contract_size * pip (converted to USD)
swap nightly      = lots * (side==buy ? swap_long : swap_short)  at 00:00 UTC, x3 on Wednesday (fx convention)
```

Price feed

Topic	Decision
v1 source	Binance WS (bookTicker) for crypto: BTCUSDT, ETHUSDT, LTCUSDT, XRPUSDT, XMRUSDT -> BTCUSD etc. Free, no key.
FX / metals / indices / stocks	Twelve Data WS (paid, launch) or Finnhub. Adapter interface: connect(), onTick(symbol,bid,ask,ts), reconnect with backoff. Until paid key exists, FX runs on a demo synthetic walker clearly labelled DEMO FEED.
Normalization	feed_symbol -> symbols.id; bid/ask; if feed gives last only, bid=last-spread/2, ask=last+spread/2 using symbols.spread_markup.
Redis keys	price:{S} hash {bid,ask,ts}; pubsub channel ticks (json); candle:live:{S}:{tf} hash; pos:{id} floating pnl; acct:{id} last snapshot.
Candles	Worker aggregates ticks per tf in memory, writes closed bars to candles table, emits candle WS events. Backfill on boot from REST (Binance klines) for 1500 bars per tf.
Staleness	If no tick for symbol in 10s -> mark is_open=false for trading (quote stale), UI shows grey price. Prevents trading on frozen prices.
Market hours	symbol_sessions define open windows; crypto 24/7; FX Sun 22:00 - Fri 22:00 UTC; stocks exchange hours. Orders outside = rejected 'market closed'.
Throttle	WS tick emit per symbol max 4/s; risk engine processes every tick but batches account snapshots at 2/s.

Worker loop (pseudo)

```
on tick(S, bid, ask, ts):
  redis.hset(price:S, {bid, ask, ts}); redis.publish(ticks, ...)
  candles.update(S, mid, ts)
  for pos in openPositionsBySymbol[S]:            # in-memory index refreshed from DB every 5s + on events
      pos.floating = pnl(pos, bid, ask); touchedAccounts.add(pos.account_id)
      if hitSL(pos,bid,ask) or hitTP(pos,bid,ask): enqueue close(pos, reason)
  for ord in pendingOrdersBySymbol[S]:
      if triggered(ord,bid,ask): enqueue fill(ord)
  every 500ms: for acc in touchedAccounts: snapshot = computeAccount(acc)
      emit ws account; if level<=stopOut: enqueue stopOut(acc) ; elif level<=marginCall: notify once/hour
stopOut(acc): loop { pos = worstLoss(open(acc)); close(pos,'stopout'); recompute; } while level<=stopOut && open>0
```

Concurrency: all closes/fills go through one BullMQ queue per account (jobId = accountId lock) so two closes never race on the same balance.

09 SECURITY

Area	Rule
Passwords	argon2id; policy 8-15+ with upper/lower/digit/special (match Spova hint); breach check optional
Cookies	access (15m) + refresh (7d) httpOnly Secure SameSite=Lax; CSRF: double-submit token header X-CSRF on state-changing requests from web
RBAC	Nest guards: @Roles('admin'), ownership guard on /me/accounts/:id (account.user_id == req.user.id)
Rate limits	login 5/min/IP + 20/hour/email; register 3/hour/IP; forgot 3/hour/email; orders 10/s/account; global 300/min/IP via Redis
Validation	zod/class-validator on every DTO; decimals parsed as strings -> Decimal.js; reject NaN/negative/over-precision
Uploads	presigned PUT only; mime whitelist jpg/png/pdf; max 10MB; random key; never public; signed GET 5 min; (later) virus scan

Area	Rule
Money	Decimal.js everywhere; DB NUMERIC; row lock on account; ledger append-only (DB role without UPDATE/DELETE on ledger_entries, admin_audit)
Idempotency	Idempotency-Key on POST orders/payments/credit/debit; cached response 24h
Audit	every admin mutation -> admin_audit before/after; login events -> sessions
Headers	helmet, CORS allowlist (web origin only), HSTS, no x-powered-by
Secrets	.env on server only; never in repo; rotate JWT secret = logs everyone out
Logging	pino JSON, request id, no PII in logs; errors to Sentry at launch
Backups	nightly pg_dump to R2, 30-day retention, restore tested once before 16 Oct
2FA	schema ready (users.totp_secret nullable), UI later

10 JOBS + QUEUES (BullMQ on Redis)

Queue	Trigger	Work	Retry
`trading:{accountId}`	fill / close / stop-out events	serialized money changes per account	3, exp backoff, dead-letter alert
`risk-engine`	ticks (in-process in worker)	pnl, SL/TP, margin, stop-out	n/a
`pending-orders`	ticks	trigger limit/stop	n/a
`swap`	cron 00:00 UTC	accrue swap per open position, ledger type=swap	1
`candles:flush`	bar close	write closed bars	3
`mail`	events	welcome, reset, kyc result, payment result, margin call	5
`statement-pdf`	user request	ledger + closed positions -> PDF -> R2 -> signed URL	2
`sessions:cleanup`	cron hourly	delete expired sessions / tokens	1
`candles:retention`	cron daily	delete old m1/m5	1
`backup`	cron 02:00	pg_dump | gzip | rclone R2	alert on fail
`feed:health`	every 10s	mark stale symbols, alert if feed down > 60s	n/a

11 INFRA + DEPLOY

```
services (docker-compose.yml)
  caddy      : 80/443 -> web:3000 (<domain>), api:4000 (api.<domain>), auto TLS (or Cloudflare origin cert)
  web        : next start, NODE_ENV=production
  api        : node dist/main.js  (REST + WS)          replicas 1 (sticky not needed with 1)
  worker     : node dist/worker.js                     exactly 1 instance (price + risk)
  postgres   : postgres:16, volume pgdata, shm 256m
  redis      : redis:7 --appendonly yes, volume
  (optional) pgadmin/adminer bound to localhost only

domains
  <domain>            marketing + /app + /trade + /admin   (one Next app)
  api.<domain>        REST + /ws
  staging.<domain>    same compose on a second port/profile for demos 9 & 13 Oct

server
  Hetzner CX33 (4 vCPU / 8 GB) Ubuntu 24.04, ufw 22/80/443, fail2ban, docker + compose plugin
  swap 2 GB, unattended-upgrades, time sync (chrony) - trading needs correct clocks

ci (GitHub Actions)
  on PR: pnpm install, lint, typecheck, prisma validate, build web+api+worker
  on main: build images, ssh deploy: docker compose pull && up -d && prisma migrate deploy

monitoring v1
  /health + uptime ping (free UptimeRobot), docker logs -> journald, pino; Sentry at launch
  alerts: feed down > 60s, queue dead-letter > 0, disk > 80%, backup failed
```

12 ENVIRONMENT VARIABLES

Var	Used by	Example / note
`DATABASE_URL`	api worker	postgresql://spova:***@postgres:5432/spova
`REDIS_URL`	api worker	redis://redis:6379
`JWT_ACCESS_SECRET / JWT_REFRESH_SECRET`	api	64 random bytes each
`JWT_ACCESS_TTL / JWT_REFRESH_TTL`	api	15m / 7d
`COOKIE_DOMAIN`	api	.
`WEB_ORIGIN`	api	https:// (CORS allowlist)
`NEXT_PUBLIC_API_URL / NEXT_PUBLIC_WS_URL`	web	https://api. / wss://api.
`R2_ACCOUNT_ID / R2_ACCESS_KEY_ID / R2_SECRET_ACCESS_KEY / R2_BUCKET`	api worker	private bucket kyc

Var	Used by	Example / note
MAIL_PROVIDER / RESEND_API_KEY / SMTP_*	api worker	resend | smtp
MAIL_FROM / ADMIN_ALERT_EMAIL	api worker	no-reply@
FEED_BINANCE_ENABLED	worker	true
FEED_TWELVEDATA_KEY	worker	empty until launch -> synthetic DEMO FEED for fx
DEMO_START_BALANCE	api	10000
KYC_REQUIRED_FOR_LIVE / KYC_REQUIRED_FOR_WITHDRAW	api	true / true
QUOTE_STALE_SECONDS	worker api	10
RATE_LIMIT_*	api	per section 09
SENTRY_DSN	all	optional
SEED_ADMIN_EMAIL / SEED_ADMIN_PASSWORD	api seed	first superadmin; change after first login

13 MILESTONE ACCEPTANCE (what must exist, demoable on staging)

9 Oct | First demo (30%)

- Domain live, Caddy TLS, compose up on server, CI green, Prisma migrations applied, seed (admin, symbols, risk config).
- Marketing: home + at least Forex/Crypto market pages + trading-account tiers + login/register entry points, responsive, Spova-like theme.
- Auth: register (creates demo account), login, logout, refresh, forgot/reset email actually sent.
- Client shell /app: rail, dashboard cards with real demo balance from DB, accounts page, profile (view/edit), password change.
- Admin shell /admin: login as admin, clients list, client detail (profile + accounts), credit/debit demo account -> ledger visible to client.
- WS connected: account snapshot updates when admin credits.

13 Oct | Second demo (30%, cumulative 80%)

- KYC: upload POI/POR to R2, statuses, admin approve/reject with reason, user flags update, email.
- Payments: deposit + withdraw requests, history list, admin approve/decline -> ledger; rules (demo no withdraw; KYC gate on live).
- Trading: symbols list, Binance live ticks, candles (1m..1d), chart, market buy/sell, positions table with live P/L, SL/TP edit, close/partial close, pending limit/stop, history.
- Risk: margin preview on ticket, reject on insufficient margin, margin call toast, stop-out works (test with high leverage demo).
- Statement PDF export. Help tickets create/list. Notification prefs.
- Admin: payments queue, kyc queue, trades page (live), force close.

16 Oct | Whole project end (20%)

- Admin complete: freeze user, block trading, change leverage/tier/status, symbols admin (enable/markup/limits), risk config per tier, audit log, tickets inbox, staff management, dashboard KPIs.
- Live account: open live (KYC gated), same trading screens, withdrawal gated.
- Remaining marketing pages (commodities, indices, shares, metals, about, FAQ, legal, contact), footer risk warning, 404.
- Hardening: rate limits on, helmet, backups running + one restore test, Sentry/uptime, seed admin password rotated.
- Handover: repo access, .env.example, README run/deploy, admin credentials, this document updated with any deltas.

14 DO-NOT-MISS LIST (things that break broker software)

Area	Gotcha	Our answer
Money math	float rounding, 0.1+0.2	Decimal.js + NUMERIC; round half-even to 2dp only at ledger write
Race conditions	two closes / admin debit while stop-out	row lock FOR UPDATE + per-account queue
Price staleness	feed drops, users trade on frozen price	10s staleness -> trading disabled for symbol, UI grey
Spread	buy and sell at same price = free money	buy@ask, sell@bid always; markup per symbol
Lot rounding	0.013 lots	lot_step validation, min/max per symbol
Digits	EURUSD 5dp vs BTC 2dp vs JPY 3dp	symbols.digits drives UI + validation + pip
Timezones	server local time in candles	everything UTC; UI shows user tz
Market hours	trading FX on Saturday	symbol_sessions + holidays; crypto 24/7
Leverage change	admin lowers leverage -> instant stop-out	pre-check + warning
Credit vs balance	bonus withdrawn as cash	credit column separate; withdraw only from balance; credit removed by admin
Demo vs live	demo withdraw request	type check server-side, not just UI
Negative balance	gap through stop-out	clamp balance at 0 on stop-out close, ledger adjustment (negative balance protection as marketed)
Idempotency	double-click Buy = two positions	Idempotency-Key + client_ref unique
Session theft	stolen refresh cookie	rotation + reuse detection
KYC privacy	public bucket / predictable URLs	private R2 + signed URLs + random keys
Audit	who gave 1M to a demo	admin_audit on every mutation, append-only
Email deliverability	reset mail in spam	SPF/DKIM/DMARC on domain before 9 Oct

Area	Gotcha	Our answer
Clock drift	candles misaligned, JWT exp issues	chrony on server
Backups	never restored	restore drill before 16 Oct
Legal copy	promising regulation we do not have	placeholders clearly marked; risk warning footer on every page
Mobile	terminal unusable on phone	responsive /app; /trade tablet+ first, mobile basic
SEO	client pages indexed	noindex on /app /trade /admin; sitemap for marketing
i18n	hardcoded strings	next-intl keys from day 1, en only shipped
Seed	no symbols -> empty chart	seed 25+ symbols with contract specs